# Gym Tracker Pro
Requirements Specification

---

## Overview
Small gyms and personal trainers need a lightweight digital tool to manage their members, maintain an exercise catalogue, and record workout sessions. Currently most small gyms rely on paper logs or generic spreadsheets, leading to:

- No traceability of a member's performance over time.
- No quick way to identify which exercises are most often performed or most improved upon.
- No structured view of session frequency or training intensity per member.

GymTrackerPro is a web-based application with two distinct user interfaces, one for gym administrators (personal trainers) and one for gym members, enabling:

1. Gym administrators to manage the members (CRUD).
2. Gym administrators to manage a catalogue of exercises (CRUD).
3. Gym administrators to log workout sessions that link a member to a set of exercises with recorded metrics (CRUD).
4. A cross-entity operation - log a complete workout session for a member, associating one or more exercises with performance data in a single atomic action (uses all 3 entities).
5. The generation of a progress report for a selected member showing the evolution of their training over time with total volume, session frequency, and per-exercise improvement.

## Primary Actors

| Role | Description |
|---|---|
| Gym Administrators | A personal trainer who works at the gym. Has full access to all data and operations. |
| Gym Members | A gym customer. Can only view their own profile, workout sessions, and progress report. |

## Use Cases

| ID | As a(n)... | I need to... | So that... |
|---|---|---|---|
| UN-01 | Gym Administrator | Add, edit, suspend, unsuspend, delete, and view gym members | The member base stays accurate and historical data is preserved |
| UN-02 | Gym Administrator | Add, edit, archive, unarchive, delete, and view exercises with | Trainers can pick from standardised exercises when |

| | | | |
|---|---|---|---|
| | | muscle group and used equipment information | logging sessions |
| UN-03 | Gym Administrator | Add, edit, view, and delete a workout session for a gym member | No data is lost and sessions are recorded atomically |
| UN-04 | Gym Administrator | Generate a progress report for any gym member, filtered by date range | Training effectiveness can be assessed and shared with the member |
| UN-05 | Gym Member | View personal training history and progress report | I can track my own improvement without other members accessing my data. |
| UN-06 | Gym Member | View personal profile information | I can verify the data the gym holds about me |
| UN-07 | Gym Administrator | Be the only one who can create, edit, or delete records | Data integrity is maintained |

## Project Scope

In scope:

- Custom session-based authentication.
- Two roles - gym administrator and gym member, with clearly separated views and permissions.
- Full CRUD for - members, exercises, workout sessions (and its child session exercises).
- Cross-entity workflow - log workout session (links Member, Exercise, WorkoutSession).
- Report - member progress report (filterable by date range).

Out of scope:

- Self-registration by members - gym administrators create all accounts.
- Payment processing or subscription management.
- Exercise video or image embedding.
- Mobile native app.
- Real-time notifications or messaging.
- Integration with wearables or third-party fitness APIs.

Assumptions:

- One deployment instance per gym.
- At least one gym administrator account is seeded at initialization.
- PostgreSQL database is provisioned and accessible before first launch.
- Users access the system via a modern browser (Chrome, Firefox, Edge).
- All weights are recorded in kilograms (metric system).

## Initial State

| State | Description |
| --- | --- |
| Database | All pending migrations are applied. All tables (Exercise, Member, WorkoutSession, SessionExercise) are initially empty. |
| Members | A seed script then creates one gym administrator account with the following credentials: admin@gymtrackerpro.com (email) / Admin1234* (password) |

## Functional Requirements

Requirements are organized by module. Each requirement is uniquely identified. Priority values follow the MoSCoW framework: MUST, SHOULD, COULD.

### Authentication (AUTH)

| ID | Requirement | Priority |
| --- | --- | --- |
| AUTH-01 | The system shall allow both gym administrators and gym members to login with email (trimmed of surrounding whitespaces) and password. | MUST |
| AUTH-02 | The system shall hash passwords using bcrypt.js with 12 rounds of salting before storing them. | MUST |
| AUTH-03 | The system shall create a server-side session on successful login and destroy it on logout. | MUST |
| AUTH-04 | The system shall redirect unauthenticated users to the login page for any protected route. | MUST |
| AUTH-05 | The system shall direct gym administrators to the administrator dashboard and gym members to the member dashboard after login. | MUST |
| AUTH-06 | Gym administrator accounts are specified at the seeding phase, whilst Gym member accounts shall only be created by a gym administrator. | MUST |
| AUTH-07 | The system shall deny access to all protected member routes for any gym member whose account has been suspended, redirecting them to the unauthorized page. | MUST |

### Member Management - Admin View (MEM)

| ID | Requirement | Priority |
| --- | --- | --- |
| MEM-01 | The system shall allow a gym administrator to create a Member account with: full name (8-64 characters)(trimmed of surrounding whitespaces), email (simplified RFC 5321)(trimmed of surrounding whitespaces), date of birth (DD-MM-YYYY), phone number (simplified E.164)(trimmed of surrounding whitespaces), and membership start | MUST |

| | | |
|---|---|---|
| | date (DD-MM-YYYY). | |
| MEM-02 | The system shall auto-generate a temporary password (16 characters, with minimum one uppercase character, one number, and one special character) for new gym member accounts, displayed once to the gym administrator. | MUST |
| MEM-03 | The system shall allow a gym administrator to view a paginated (10 elements per page), searchable list (by full name or email) of all gym members, sorted in ascending order by full name. | MUST |
| MEM-04 | The system shall allow a gym administrator to view the full detail (except password) page of any gym member. | MUST |
| MEM-05 | The system shall allow a gym administrator to update any field of a gym member record (except the password). | MUST |
| MEM-06 | The system shall allow a gym administrator to suspend and unsuspend gym members (isActive attribute boolean flag inside gym member record), preserving all historical session data. | SHOULD |
| MEM-07 | The system shall prevent duplicate email addresses across Member records. | SHOULD |

### Member Self-View - Member View (MSV)

| ID | Requirement | Priority |
|---|---|---|
| MSV-01 | The system shall allow a gym member to view their own profile. | MUST |
| MSV-02 | The system shall allow a gym member to view their own workout sessions (read-only) as a paginated list (10 elements per page), sorted in descending order by date. | MUST |
| MSV-03 | The system shall prevent a gym member from viewing any other member's data. | MUST |
| MSV-04 | The system shall allow a gym member to change their own password (8-64 characters, minimum one uppercase character, one number, and one special character). | SHOULD |

### Exercise Management - Admin View (EXM)

| ID | Requirement | Priority |
|---|---|---|
| EXM-01 | The system shall allow a gym administrator to create an exercise with: name (8-64 characters)(trimmed of surrounding whitespaces), description (0-1024 characters)(trimmed of surrounding whitespaces), muscle group (enum - “CHEST”, “SHOULDERS”, “ARMS”, “BACK”, “CORE”, | MUST |

| | | |
|---|---|---|
| | “LEGS”), and equipment type (enum - “CABLE”, “DUMBBELL”, “BARBELL”, “MACHINE”). | |
| EXM-02 | The system shall allow a gym administrator to view a paginated (10 elements per page), searchable list (by name) of all exercises, sorted in ascending order by exercise name. | MUST |
| EXM-03 | The system shall allow a gym administrator to update any exercise record. | MUST |
| EXM-04 | The system shall allow a gym administrator to archive and unarchive an exercise (isActive attribute boolean flag inside exercise record), removing it from new workout session forms while preserving historical references. | MUST |
| EXM-05 | Exercise names shall be unique within the system. | SHOULD |
| EXM-06 | Members shall be able to view the paginated (10 elements per page), searchable list (by name) of all exercises, sorted in ascending order by name, in read-only mode. | SHOULD |

**Workout Session Management - Admin View (WSM)**

| ID | Requirement | Priority |
|---|---|---|
| WSM-01 | The system shall allow a gym administrator to create a workout session for a gym member, specifying the date (DD-MM-YYYY), duration in minutes (Integer, 0-180), and optional notes(0-1024 characters)(trimmed of surrounding whitespaces). | MUST |
| WSM-02 | During session creation, the gym administrator shall be able to add one or more session exercises, each specifying the exercise, number of sets (Integer, 0-6), reps per set (Integer, 0-30), and weight in kg (Float, 0.0-500.0). | MUST |
| WSM-03 | A workout session shall require at least one session exercise before it can be saved. | MUST |
| WSM-04 | The creation of a workout session and all its session exercises shall be atomic (single database transaction). | MUST |
| WSM-05 | The system shall allow a gym administrator to view a paginated list (10 elements per page) of workout sessions for a given member (select), ordered in descending order by date. | MUST |
| WSM-06 | The system shall allow a gym administrator to edit the date, duration, notes and workout session exercise details of an existing workout session. | SHOULD |

| | | |
|---|---|---|
| WSM-07 | The system shall allow a gym administrator to add, view, update, and remove session exercises from an existing workout session. | SHOULD |
| WSM-08 | The system shall allow a gym administrator to delete a workout session, cascading the deletion to its session exercises. | SHOULD |

### Report Generation - Member Progress Report (RPG)

| ID | Requirement | Priority |
|---|---|---|
| RPG-01 | The system shall provide a progress report page per gym member, filterable by date range (DD-MM-YYYY). | MUST |
| RPG-02 | The report shall display: total sessions in period, total training volume (sum of sets × reps × weight), and average session duration. | MUST |
| RPG-03 | The report shall display a per-exercise breakdown: exercise name, muscle group, workout sessions count in which it is present, total sets performed, total reps performed, and weight volume lifted in the period. | MUST |
| RPG-04 | The report shall display a chronological table of all sessions in the period with: date, duration, and the exercises performed. | MUST |
| RPG-05 | The report data shall be computed server-side. | MUST |
| RPG-06 | A gym administrator shall be able to generate the report for any gym member. | MUST |
| RPG-07 | A gym member shall only be able to generate the report for themselves. | MUST |

## Non-Functional Requirements (Out of the scope of testing)

| ID | Category | Requirement |
|---|---|---|
| NFR-01 | Performance | All pages shall load within 3 seconds on a standard broadband connection (~10mbps). |
| NFR-02 | Usability | The system shall be usable on screen widths >= 375px (all components shall be visible inside the viewport). |
| NFR-03 | Security | All form inputs shall be validated both client-side and server-side. |
| NFR-04 | Security | Passwords (excluding temporary ones) shall never be stored or transmitted in plain text. |
| NFR-05 | Reliability | Server errors shall be logged server-side without exposing stack traces to the end user. |
| NFR-06 | Maintainability | The database schema shall be managed via Prisma migrations. |

| | | |
|---|---|---|
| NFR-07 | Maintainability | The codebase shall follow the directory structure and naming conventions defined in the design document. |
| NFR-08 | Portability | The application shall be containerized using Docker Compose, with separate services for the application, database, and migration processes. |